

stock_trading_agent

Multi-Agent AI Trading System — Architecture

A team of specialised LLM-backed agents that analyse, debate, size under a risk mandate, and decide — then prove it on a walk-forward backtester.

Educational / research only — not investment advice. Runs fully offline and deterministically by default.

1. System Overview

stock_trading_agent decomposes the question “should I trade this asset?” the way an investment desk does. Specialised agents form independent views, two researchers debate the bull and bear cases, a risk officer sizes the position under a hard mandate, and a portfolio manager issues the final decision. The identical decision function plugs straight into a walk-forward backtester for honest evaluation against baselines.

The system is organised in small, typed, independently testable layers. It runs offline and deterministically out of the box and upgrades to live LLMs and live market data by configuration only.

2. Decision Pipeline

Market data feeds five independent analysts (run concurrently). Their evidence drives an adversarial debate that yields a net conviction; risk translates that into a vol- and Kelly-aware position with ATR stops; the portfolio manager issues the bounded decision and resolved trades feed reflection memory.

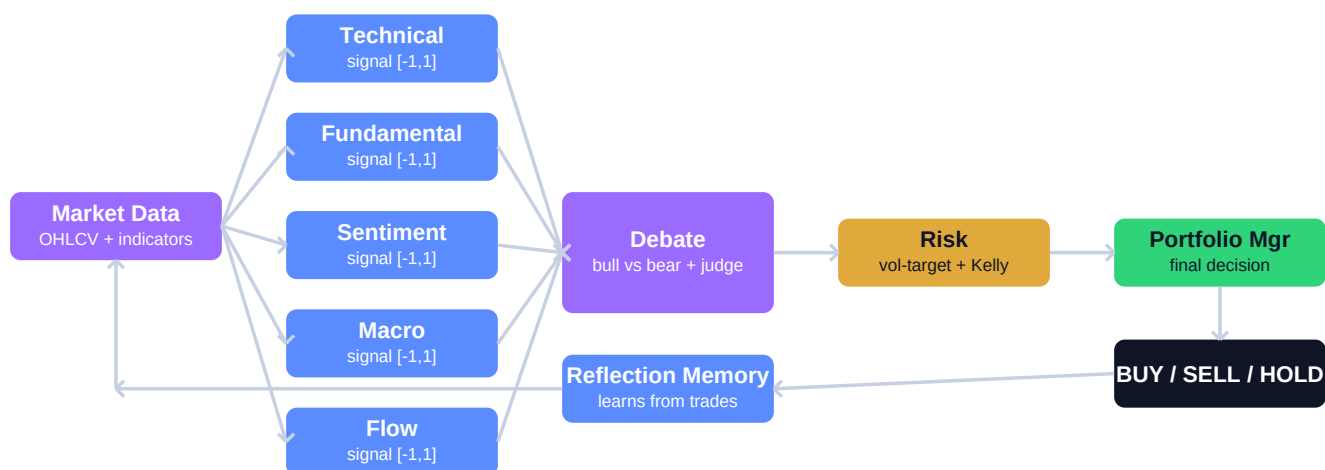


Figure 1 — End-to-end multi-agent decision pipeline with risk veto and reflection-memory feedback loop.

3. Layered Architecture

Each layer exposes a small contract, so any layer can be tested or replaced in isolation. The service layer is the seam between business logic and transport: the CLI and HTTP API both build on it.

Layer	Package	Responsibility
Configuration	config.py	Typed settings from env / .env (pydantic)
Data	data/	Market data providers + technical indicators
LLM abstraction	llm/	Provider-agnostic chat with offline fallback
Agents	agents/	Analysts, debate, risk, portfolio manager
Memory	memory/	File-backed reflection memory
Orchestration	orchestration/	DAG engine + end-to-end pipeline
Backtest	backtest/	Portfolio, engine, metrics, strategies
Service	service.py	Framework-agnostic JSON facade
Interfaces	cli.py, api/, web/	CLI, REST API, web dashboard

4. Orchestration Engine

A dependency-graph (DAG) engine validates the wiring (detecting cycles and missing dependencies), computes dependency levels, and runs each level — executing mutually-independent nodes concurrently on a thread pool. Updates are merged back in a stable, name-sorted order, so a parallel run is identical to a sequential one. This is a real latency win for live-LLM runs while preserving determinism.

5. Determinism & the LLM Abstraction

Every agent first computes a complete heuristic decision from the data; the LLM is asked only to improve it. Offline, the heuristic is the decision, so the whole system is reproducible with no API key. A missing provider key degrades gracefully back to offline mode rather than crashing.

6. Risk & Position Sizing

- **Volatility targeting** — scale exposure so expected position volatility matches a target, instead of fixed bet sizes.
- **Fractional Kelly** — blend in a tempered Kelly bet (edge / variance) governed by a configurable fraction.
- **ATR-based stops** — stop-loss / take-profit widths adapt to each name's true range.
- **Hard veto** — the risk officer can reject a trade and cap any single position; the portfolio manager cannot override it.

7. Strategies & Backtesting

The walk-forward engine only ever shows a strategy the history available up to each decision date (no look-ahead), marks to market daily, and honours stops. Metrics: total return, CAGR, Sharpe, Sortino, max drawdown, Calmar and win rate.

Strategy	Idea
multi_agent	Full debate + risk pipeline
momentum	Time-series momentum, vol-targeted
mean_reversion	RSI dip-buyer / strength-seller
bollinger_breakout	Trade closes beyond the bands
ensemble	Confidence-weighted blend of technicals
sma_crossover / buy_and_hold	Classic baselines

8. Interfaces & Extensibility

- **CLI** — analyze, backtest, serve, config, version.
- **REST API** — FastAPI with typed request models and OpenAPI docs.
- **Web dashboard** — buildless SPA (vanilla JS + Chart.js).
- **Extend** — new analysts implement BaseAgent; new strategies implement the Strategy protocol; new data/LLM providers implement their interfaces.